1. **Задание**

   Разработать и отладить мобильное приложение, обеспечивающее подключение и работу с базой данных (например, SQLite для ОС Android). Приложение должно реализовывать создание, чтение, изменение и удаление отдельных записей

2. **Ход работы**

   Для выполнения работы была использована библиотека Room от Google для работы с БД при помощи ORM и DAO

   На следующий рисунках 1-4 показана работа приложения.

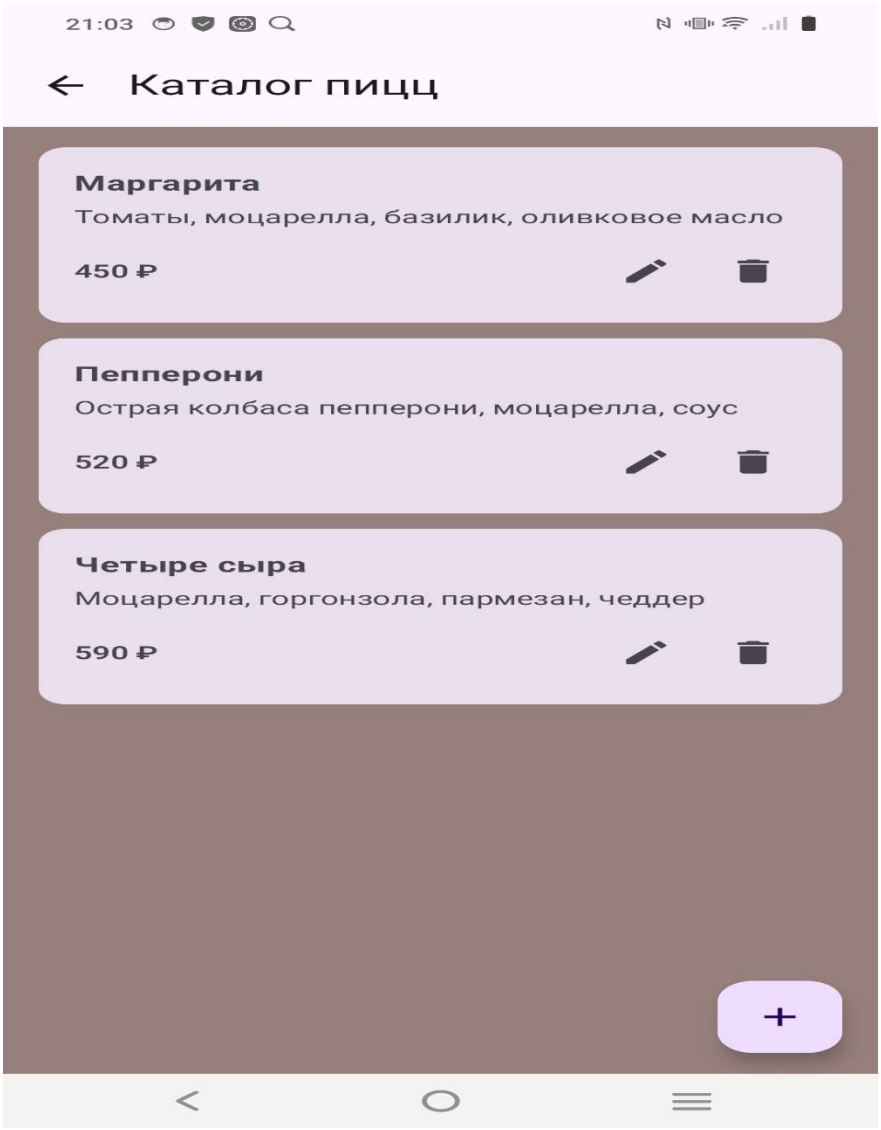


Рисунок 1 – Список данных

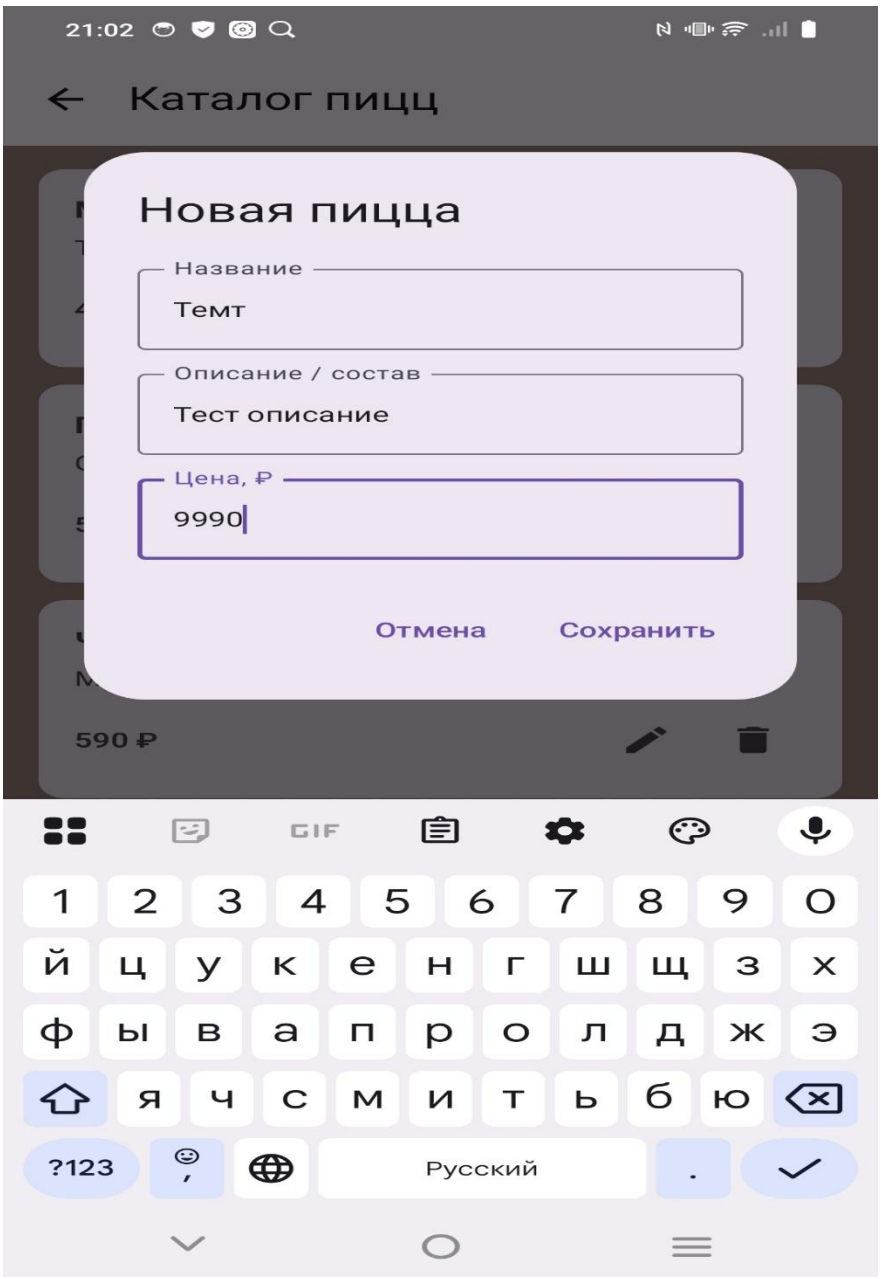


Рисунок 2 – Добавление нового элемента

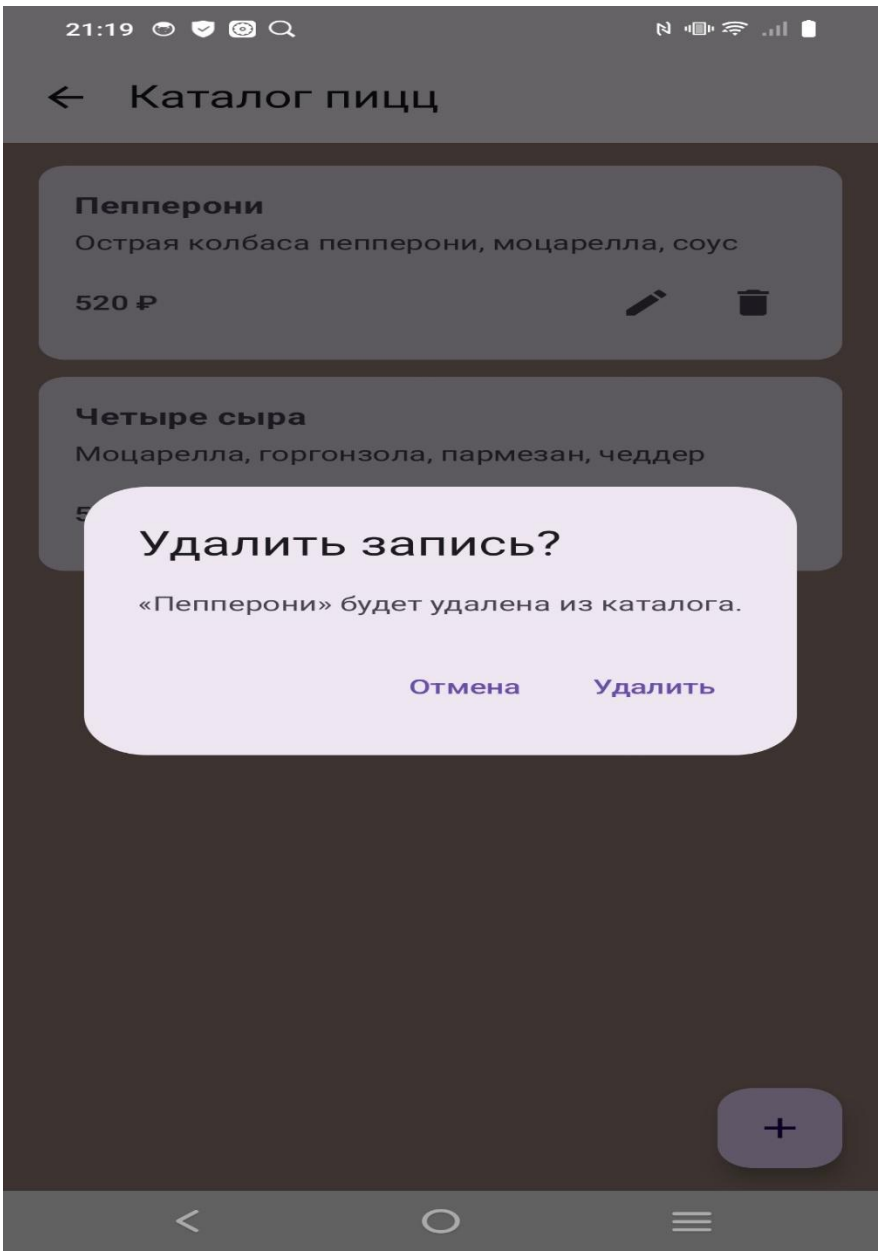


Рисунок 3 – Удаление элемента



Рисунок 4 – Список данных после манипуляций

Новый код приложения приведён далее

Catalog Acticity

```
package com.example.lab3

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.LocalActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
```

```
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.filled.Add
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material.icons.filled.Edit
import androidx.compose.material3.AlertDialog
import androidx.compose.material3.Card
import androidx.compose.material3.CardDefaults
import androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.FloatingActionButton
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.LaunchedEffect
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalContext
```

```
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import com.example.lab3.data.AppDatabase
import com.example.lab3.data.PizzaEntity
import com.example.lab3.ui.theme.Lab3Theme
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class CatalogActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Lab3Theme {
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background,
                ) {
                    CatalogScreen()
                }
            }
        }
    }
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun CatalogScreen() {
    val context = LocalContext.current
    val activity = LocalActivity.current
    val database = remember { AppDatabase.get(context) }
    val dao = remember(database) { database.pizzaDao() }
    val scope = rememberCoroutineScope()
```

```
var pizzas by remember { mutableStateOf<List<PizzaEntity>>(emptyList()) }

var showEditor by remember { mutableStateOf(false) }
var editingPizza by remember { mutableStateOf<PizzaEntity?>(null) }
var nameField by remember { mutableStateOf("") }
var descField by remember { mutableStateOf("") }
var priceField by remember { mutableStateOf("") }

var pizzaToDelete by remember { mutableStateOf<PizzaEntity?>(null) }

LaunchedEffect(dao) {
    withContext(Dispatchers.IO) {
        if (dao.count() == 0) {
            listOf(
                PizzaEntity(
                    name = "Маргарита",
                    description = "Томаты, моцарелла, базилик, оливковое масло",
                    priceRub = 450,
                ),
                PizzaEntity(
                    name = "Пепперони",
                    description = "Острая колбаса пепперони, моцарелла, соус",
                    priceRub = 520,
                ),
                PizzaEntity(
                    name = "Четыре сыра",
                    description = "Моцарелла, горгонзола, пармезан, чеддер",
                    priceRub = 590,
                ),
            ).forEach { dao.insert(it) }
        }
    }
    dao.observeAll().collect { pizzas = it }
}
```

```
fun openCreate() {
    editingPizza = null
    nameField = ""
    descField = ""
    priceField = ""
    showEditor = true
}

fun openEdit(pizza: PizzaEntity) {
    editingPizza = pizza
    nameField = pizza.name
    descField = pizza.description
    priceField = pizza.priceRub.toString()
    showEditor = true
}

fun closeEditor() {
    showEditor = false
    editingPizza = null
}

fun savePizza() {
    val name = nameField.trim()
    if (name.isEmpty()) return
    val price = priceField.trim().toIntOrNull() ?: return
    val desc = descField.trim()
    scope.launch {
        withContext(Dispatchers.IO) {
            val entity = editingPizza?.copy(
                name = name,
                description = desc,
                priceRub = price,
            ) ?: PizzaEntity(
                name = name,
                description = desc,
```

```
                priceRub = price,
            )
            if (editingPizza == null) {
                dao.insert(entity)
            } else {
                dao.update(entity)
            }
        }
        closeEditor()
    }
}

Scaffold(
    topBar = {
        TopAppBar(
            title = { Text("Каталог пицц") },
            navigationIcon = {
                IconButton(onClick = { activity?.finish() }) {
                    Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription =
"Назад")
                }
            },
        )
    },
    floatingActionButton = {
        FloatingActionButton(onClick = { openCreate() }) {
            Icon(Icons.Default.Add, contentDescription = "Добавить пиццу")
        }
    },
) { innerPadding ->
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(innerPadding)
            .padding(horizontal = 16.dp),
```

```
            verticalArrangement = Arrangement.spacedBy(12.dp),
        ) {
            item { Spacer(Modifier.height(4.dp)) }
            items(pizzas, key = { it.id }) { pizza ->
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.surfaceVariant,
                    ),
                ) {
                    Column(Modifier.padding(16.dp)) {
                        Text(pizza.name, style = MaterialTheme.typography.titleMedium,
fontWeight = FontWeight.Bold)
                        Spacer(Modifier.height(4.dp))
                        Text(pizza.description, style = MaterialTheme.typography.bodyMedium)
                        Spacer(Modifier.height(8.dp))
                        Row(
                            modifier = Modifier.fillMaxWidth(),
                            horizontalArrangement = Arrangement.SpaceBetween,
                            verticalAlignment = Alignment.CenterVertically,
                        ) {
                            Text(
                                "${pizza.priceRub} ₽",
                                style = MaterialTheme.typography.titleSmall,
                                fontWeight = FontWeight.SemiBold,
                            )
                            Row {
                                IconButton(onClick = { openEdit(pizza) }) {
                                    Icon(Icons.Default.Edit, contentDescription = "Изменить")
                                }
                                IconButton(onClick = { pizzaToDelete = pizza }) {
                                    Icon(Icons.Default.Delete, contentDescription = "Удалить")
                                }
                            }
                        }
```

```
                }
            }
        }
        item { Spacer(Modifier.height(80.dp)) }
    }
}

if (showEditor) {
    AlertDialog(
        onDismissRequest = { closeEditor() },
        title = { Text(if (editingPizza == null) "Новая пицца" else "Изменить пиццу") },
        text = {
            Column {
                OutlinedTextField(
                    value = nameField,
                    onValueChange = { nameField = it },
                    label = { Text("Название") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth(),
                )
                Spacer(Modifier.height(8.dp))
                OutlinedTextField(
                    value = descField,
                    onValueChange = { descField = it },
                    label = { Text("Описание / состав") },
                    modifier = Modifier.fillMaxWidth(),
                )
                Spacer(Modifier.height(8.dp))
                OutlinedTextField(
                    value = priceField,
                    onValueChange = { priceField = it.filter { ch -> ch.isDigit() } },
                    label = { Text("Цена, ₽") },
                    singleLine = true,
                    modifier = Modifier.fillMaxWidth(),
                )
```

```
            }
        },
        confirmButton = {
            TextButton(onClick = { savePizza() }) { Text("Сохранить") }
        },
        dismissButton = {
            TextButton(onClick = { closeEditor() }) { Text("Отмена") }
        },
    )
}

pizzaToDelete?.let { candidate ->
    AlertDialog(
        onDismissRequest = { pizzaToDelete = null },
        title = { Text("Удалить запись?") },
        text = { Text("«${candidate.name}» будет удалена из каталога.") },
        confirmButton = {
            TextButton(
                onClick = {
                    scope.launch {
                        withContext(Dispatchers.IO) { dao.delete(candidate) }
                        pizzaToDelete = null
                    }
                },
            ) { Text("Удалить") }
        },
        dismissButton = {
            TextButton(onClick = { pizzaToDelete = null }) { Text("Отмена") }
        },
    )
}
}
```

AppDatabase

```
package com.example.lab3.data

import android.content.Context
import androidx.room.Database
import androidx.room.Room
import androidx.room.RoomDatabase

@Database(
    entities = [PizzaEntity::class],
    version = 1,
    exportSchema = false,
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun pizzaDao(): PizzaDao

    companion object {
        @Volatile
        private var instance: AppDatabase? = null

        fun get(context: Context): AppDatabase {
            return instance ?: synchronized(this) {
                instance ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "catalog_pizzas.db",
                ).build().also { instance = it }
            }
        }
    }
}
```

PizzaDao

```
package com.example.lab3.data
```

```
import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.OnConflictStrategy
import androidx.room.Query
import androidx.room.Update
import kotlinx.coroutines.flow.Flow

@Dao
interface PizzaDao {

    @Query("SELECT * FROM pizzas ORDER BY name COLLATE NOCASE ASC")
    fun observeAll(): Flow<List<PizzaEntity>>

    @Query("SELECT COUNT(*) FROM pizzas")
    suspend fun count(): Int

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insert(pizza: PizzaEntity): Long

    @Update
    suspend fun update(pizza: PizzaEntity)

    @Delete
    suspend fun delete(pizza: PizzaEntity)
}
```

PizzaEntity

```
package com.example.lab3.data

import androidx.room.Entity
import androidx.room.PrimaryKey

@Entity(tableName = "pizzas")
data class PizzaEntity(
```

```
    @PrimaryKey(autoGenerate = true) val id: Long = 0,
    val name: String,
    val description: String,
    val priceRub: Int,
)
```

3. Выводы

В процессе лабораторной работы была организована работа со встроенной в Android БД.